ACCORD

From a chat to a commitment both sides have checked

Item	Detail
Team	Big Leagues
Problem statement	LS-06: From Handshake to Contract (Informal Agreements)
Format	48-hour hackathon project, team of four
Project report	Detailed project report for the jury

Accord takes a messy business chat (English, Hindi or Hinglish), pulls out what was actually agreed, points out what was left vague or missing, and gets the other person to check every term against the original messages. Everything that happens is written into a hash-chained trail that shows if anyone changes the past. It is built so a small shopkeeper can use it without feeling like they are signing a legal contract.

Everything described as working in this report was run and tested by us. Things that are built but not yet verified on real hardware, or only planned, are listed separately in section 11.

1. Project overview

Team name: Big Leagues. Selected problem statement: LS-06, From Handshake to Contract (Informal Agreements). The brief asks for a tool that extracts terms from informal chats, flags what is ambiguous or missing, produces a plain-language agreement that both sides confirm, and keeps a tamper-evident evidence trail, all without feeling legalistic.

Executive summary. We built Accord as a working web application with a dealer side and a customer side. A dealer uploads a WhatsApp chat (plus voice notes, screenshots or call notes). Accord reads it, shows the price, quantity, delivery and payment terms it found with the exact messages behind each one, and lists what is still unsettled together with a friendly message the dealer can send to settle it. The dealer sends the agreement to the customer through a private link. The customer checks each term against their own records and only then can approve. Each step is recorded in a SHA-256 hash chain, and a PDF agreement and evidence report can be downloaded.

Numbers at a glance: about 5,000 lines of code, 9 database tables, about 55 API endpoints, 39 automated tests (all passing), and a rule-based reader that analyses a 16-message chat in under a millisecond.

2. Problem understanding

Who has this problem

Small dealers and service providers who do business on WhatsApp: printers, tailors, caterers, contractors, resellers, freelancers. Their customers are usually individuals or other small businesses. Almost none of them use written contracts, and most would not read one if given it.

The pain points we focused on

- **Terms drift during the chat.** In our sample, the price moves from 48,000 to 45,000 to 46,500, and then the order itself changes from 500 to 600 cards and the price becomes 54,000. Three weeks later each person remembers a different number.
- **Vague words are treated as agreement.** "Next week", "usual advance", "around 50k" feel clear in the chat and are useless in a dispute.
- **Important things are never discussed.** Cancellation, late delivery, defects, GST and what happens if the two sides disagree usually do not come up at all.
- **A chat is weak proof.** It can be edited, forgotten, deleted or selectively screenshotted, and nobody has confirmed that the other person agrees with the same version.
- **Legal documents scare people off.** A formal contract feels aggressive between people who have worked together for years, so it never gets made.

Why it matters

Informal deals are a large part of how small businesses in India actually work, and when they go wrong the small party usually loses, because there is nothing to point to. A light record that both sides have actually looked at is worth more than a long contract nobody reads.

Assumptions we made

- The deal is between two parties and happens mostly over chat.
- The dealer is the one who starts the process, since they are usually the one who needs proof of what was promised.
- Both parties are willing to confirm. If the other side never responds, Accord cannot make them.
- Accord records what both sides said and confirmed. It does not give legal advice, and whether a record is enforceable (including stamp duty) depends on the state and the situation.

3. Proposed solution

Accord turns the usual order of things around. Instead of asking people to write a contract, it reads what they already said and asks them to check it.

User journey

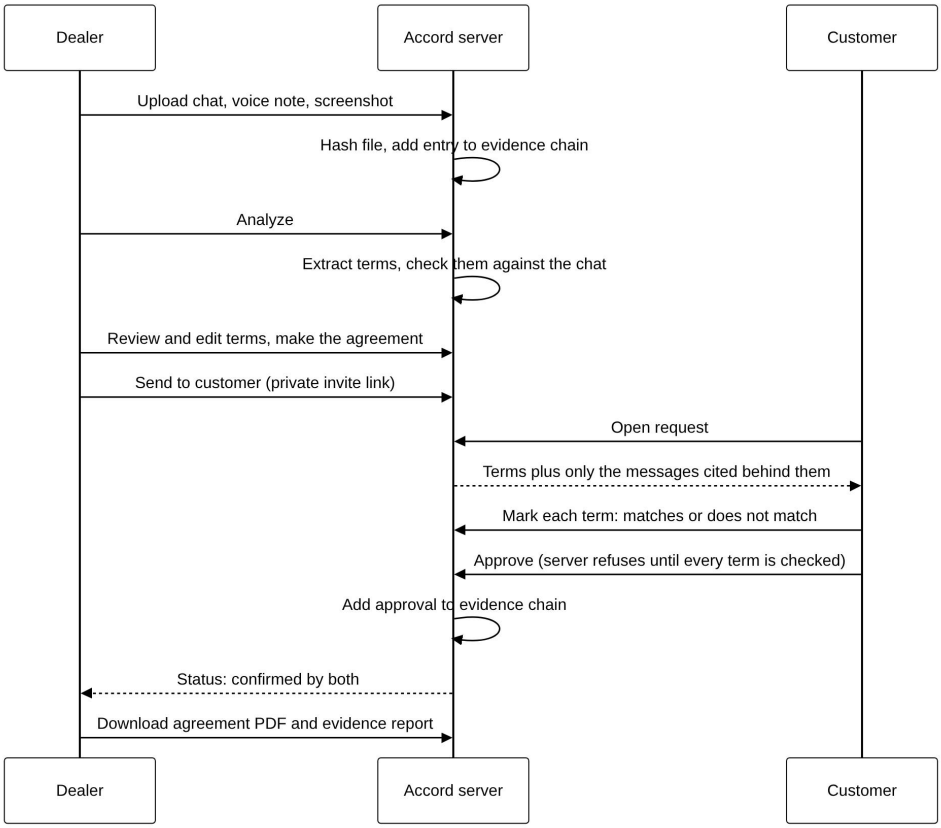


Figure 1. One deal from upload to confirmed agreement.

- **1. Add evidence.** The dealer pastes or uploads a WhatsApp export (a zip also works), and can add voice notes, screenshots, documents and call notes. Each file is fingerprinted when it is added.
- **2. Read the terms.** Accord extracts scope, price (and whether GST is included), delivery and payment. Each term shows the messages it came from. The dealer can edit anything.

- **3. Settle the gaps.** Vague phrases and missing items (advance, GST, cancellation, late delivery, defects, disputes) are listed. Each has a short, friendly message to send on WhatsApp.
- **4. Make and send the agreement.** A plain-language agreement is written from the terms and sent to the customer through a private invite link.
- **5. The customer checks.** The customer sees each term with only the messages cited behind it, and marks each one as matching or not matching. Approval stays locked until every term is checked. A mismatch becomes a change request with their note.
- **6. Keep the proof.** Every step is added to the evidence chain. The dealer and customer can download the agreement and an evidence report as PDFs.

How it answers the problem statement

Requirement in LS-06	What Accord does
Extract terms from informal chats	Rule-based reader for English/Hindi/Hinglish. Optional local model or Claude, with every model answer checked against the chat.
Flag ambiguous or missing items	Detects vague phrases and six standard gaps. Each comes with a ready-to-send nudge.
Plain-language agreement both sides confirm	Friendly-tone agreement. Customer must check every term before approval; the server enforces it.
Tamper-evident evidence trail	SHA-256 hash chain over every upload, edit, agreement and approval, with a one-click integrity check and a tamper demo.
Not legalistic	No clauses or legal jargon. The tone is a short message you would send a friend.

4. Innovation and differentiation

Tools exist that turn a pasted chat into a contract and collect a signature. We took a different route, and these are the decisions that set Accord apart.

- **Agreement on facts, not just a signature.** The customer verifies each term against the original message before approval is allowed. Signing a long document does not prove two people understood the same thing. Checking term by term gets closer to that.
- **It understands how a negotiation moves.** A price is tied to the quantity it was agreed for. When the order went from 500 to 600 cards, the old 46,500 is kept as history with the reason, and the final price becomes 54,000. A quote that the other side never accepted does not replace an accepted price; it is flagged instead.
- **It finds what is missing, not only what is there.** Six standard gaps are checked every time, and each one produces a short message the dealer can send.
- **Privacy by design.** The customer sees only the messages cited behind each term, never the dealer's whole chat. Customers are linked by a private invite link, not by matching an email or phone number, which a test confirms cannot link the wrong account.

- **Models are never trusted blindly.** Whatever a model returns is checked against the chat: message ids must exist, a price must appear in the chat, and an accepted price beats a model's different guess. The page tells the user when a correction was made.
- **Works with no AI at all.** The rule-based reader runs offline and is the fallback, so a demo cannot fail because a model was slow or an API key was missing.

5. System architecture

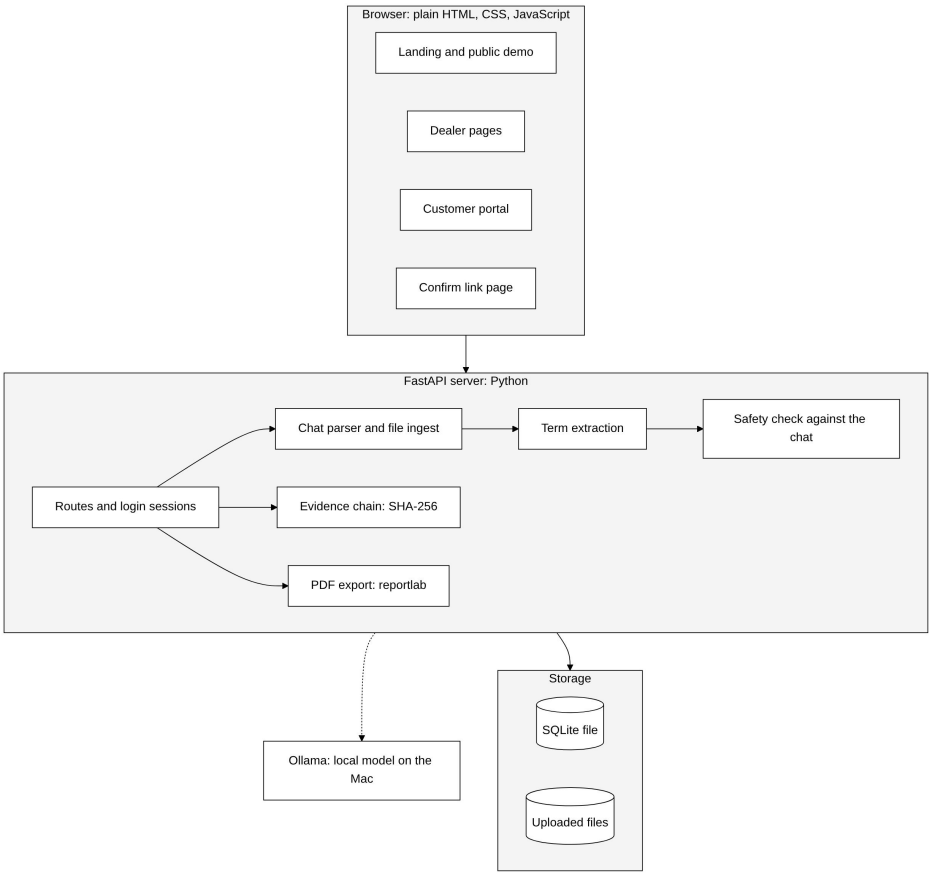


Figure 2. Main components. The dotted line to Ollama is used only when the local model is switched on.

Components

Component	Responsibility
Browser pages	Landing page with a public demo, login and sign-up, dealer dashboard and ticket workflow, customer portal, and a guest confirmation page. Plain HTML, CSS and JavaScript with no build step.
Routes and sessions	FastAPI endpoints. Login sessions use an HttpOnly cookie. Roles (dealer or customer) are enforced on the server.
Parser and ingest	Reads WhatsApp exports (including zips) and merges chat, voice notes, screenshots and call notes into one timeline.
Term extraction	Rules engine, optional local model through Ollama, optional Claude. The provider is chosen by one setting.
Safety check	Corrects or fills in anything a model got wrong, using the chat as the source of truth.
Evidence chain	Hashes every event and links it to the previous one.

Component	Responsibility
PDF export	Agreement and evidence report. DejaVu fonts are bundled so the rupee sign prints correctly.
Storage	One SQLite file for data and a folder for uploaded files.

Extraction pipeline

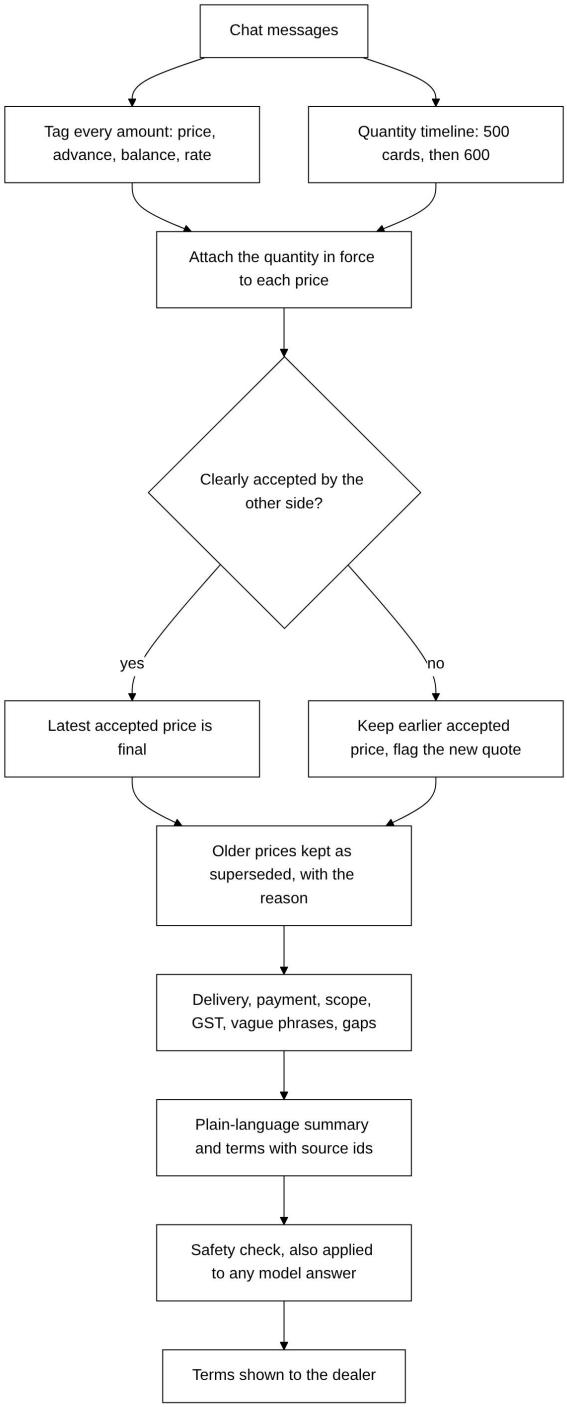


Figure 3. How the rules engine reads a negotiation and how the safety check applies to any model.

Data flow for the evidence trail

Each event (chat added, terms read, agreement written, customer response) becomes one entry holding the SHA-256 of its content and the hash of the previous entry. Changing any past entry changes its hash, which no longer matches what the next entry stored, so every later entry fails the check. The Proof tab shows the chain and the result, and a "Simulate tampering" button on the dealer side lets you watch the check fail.

Database design

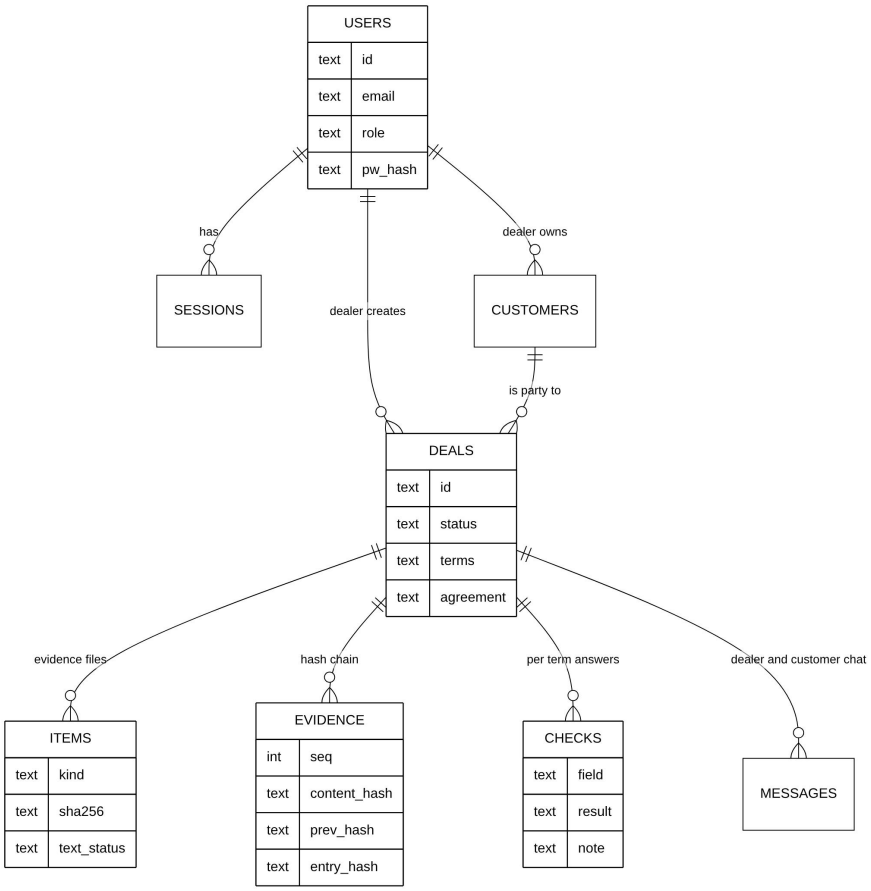


Figure 4. Main tables and how they relate.

Nine tables: users, sessions, customers, segments, deals, items (evidence files), messages (dealer and customer chat), checks (the customer's answer for each term) and evidence (the hash chain). Passwords are hashed with scrypt, and session tokens are stored hashed.

API overview

Area	Examples
Public	GET /api/sample, POST /api/demo (rules reader only, 8,000 characters, 8 requests a minute per IP), GET /healthz
Accounts	POST /api/auth/signup, login, logout, GET /api/auth/me, /api/join/{token} (invite links)
Dealer	customers, segments and tickets (create, list, update), POST /api/cases/{id}/items, POST /api/cases/{id}/analyze, POST /api/deals/{id}/agreement, POST /api/deals/{id}/send

Area	Examples
Customer	GET /api/my/tickets, PUT /api/my/tickets/{id}/checks/{field}, POST /api/my/tickets/{id}/respond, GET /api/my/dealers
Evidence and export	GET /api/deals/{id}/bundle, POST /api/verify-file, agreement.pdf and report.pdf for dealer, customer and guest link
Conversation	GET and POST /api/tickets/{id}/thread (dealer and customer chat, comments on the agreement)

6. Implementation details

Technology stack

Layer	What we used and why
Backend	Python 3 with FastAPI and Uvicorn. Small, fast to build with, validates input.
Database	SQLite through the standard sqlite3 module. One file, no setup. Fine for a demo; Postgres is the next step for real use.
Frontend	Vanilla JavaScript, one CSS file, Archivo and JetBrains Mono fonts. No framework, so nothing to build and it loads fast.
Security	scrypt password hashing, hashed session tokens, HttpOnly SameSite cookies, role checks on every route, SHA-256 evidence chain.
PDF	reportlab with bundled DejaVu fonts.
AI (optional)	Ollama running qwen2.5:7b locally; Claude through the Anthropic API; both are optional.
Testing	pytest with FastAPI TestClient and httpx; Playwright for browser checks.
Deployment	Dockerfile and start script, with notes for Render, Railway and Fly.

The extraction logic, in plain terms

The main work is in one file, app/extract.py. The rules reader works like this:

- Every amount in every message is tagged by the words right beside it: price, advance, balance or per-unit rate. "54,000 final, advance 10k" is read as two different things. Quantities like "500 cards" are not mistaken for money.
- A timeline of quantities is built. A later message such as "cards 600 chahiye" changes the quantity in force, and each price is attached to the quantity at that moment.
- A price counts as agreed when the other person repeats it with an accepting word ("theek hai 46,500 done") or replies with one and no different number. The latest agreed price is the final one. A later quote nobody accepted is flagged and does not win.
- Older prices are kept as history, and when the quantity changed the reason is recorded.

- Delivery, payment (advance, balance, mode), GST (extra or included), vague phrases and the six standard gaps are then filled in, and a short plain-language summary is written from the result.

Example output on our sample chat: "Suresh and Ravi agreed on 600 cards, 4-colour print, gold foil for ₹54,000, plus GST. The price changed 3 times along the way. ₹46,500 was for 500 cards; when the order became 600 cards, the price became ₹54,000. Delivery: 25 Oct. Advance ₹20,000; balance on delivery; paid via GPay."

How the local model is used

When switched on, the chat goes to a model running on the same Mac through Ollama. We built a small model definition (qwen2.5:7b with fixed settings), send one worked example with every request, and force the answer into a fixed JSON shape so a small model cannot return broken output. We did not fine-tune a model: on a 16 GB laptop it would be slow, and the example, fixed format and safety check do the job we needed. The answer then goes through the same safety check as any other model, with the rules result as the reference. The same code can also reach a model on another machine through a small token-protected gateway; we kept this as an option.

Other decisions worth noting

- The public landing demo never calls a paid or local model. It uses the rules reader only, so anyone on the internet can try it at no cost and cannot use our hardware.
- A sent deal is locked: the dealer can no longer edit its details, so the customer approves exactly what they saw.
- Removing an evidence item hides it but keeps the file and the trail entry, so history is never silently erased.
- The customer's report PDF only contains the messages cited behind the terms, matching what they could see on screen.

7. Achievements during the hackathon

What exists and runs today:

- A public landing page with a working demo on a sample Hinglish chat. Nothing is saved, no login needed.
- Sign-up and login for two roles, and a private invite-link system that connects a customer to a dealer.
- Dealer side: overview, tickets, customers, segments, an evidence tab, a terms tab with edit and nudges, an agreement tab, and a proof tab.
- Customer side: overview, requests, history and My dealers, with term-by-term verification and a chat with the dealer.
- A guest confirmation page opened from a link, with an OTP step (see section 11 for what this is and is not).
- Evidence ingest for pasted chats, WhatsApp zips, voice notes, screenshots, documents and call notes, merged by time.
- Tamper-evident chain with integrity check, a file checker (upload the original chat and see whether its fingerprint is in the trail) and a tamper demo.

- PDF export of the agreement and an evidence report for dealer, customer and guest link.
- A black-and-white design system used on every page.
- Deployment files and documentation: README, deployment guide, and a local AI guide.

Measurable results: 39 of 39 automated tests pass; on the sample chat the final price, scope, delivery, advance, GST and change history all come out correct; the rules reader runs in under a millisecond on that chat.

How the build went

We started with the core loop (upload chat, extract terms, agreement, hash chain) and added the customer side, accounts and invite links after. We later redesigned every page into one visual system, added PDF export, and reworked extraction after finding that our first version took the last amount in the chat as the price. That bug is the reason for the quantity timeline and the acceptance rule described above.

8. Testing and validation

We have 39 automated tests, all passing at the time of writing. They run against a real (temporary) database through the application's own API.

Test file	Tests	What it covers
test_accounts.py	19	Sign-up, login and logout; role-based redirects and API access; dealers cannot see each other's data; invite links; matching email or phone does not link accounts; customer sees only sent tickets; approval blocked until every term is checked; a mismatch becomes a change request; the public demo needs no login and saves nothing; dealer and customer chat.
test_evidence.py	8	Empty case is not ready; chat, file and note merge by time; voice note handling; removing an item keeps the trail; WhatsApp zip unpacking; a tampered stored file is caught; sent deals are locked; verify-file matches the raw upload.
test_export.py	4	PDFs for the dealer; the customer report contains only cited messages; PDF access rules; agreement PDF via guest link.
test_extract.py	7	Final price follows the quantity change; advance is not the price; an unaccepted later quote does not win; the safety check fixes a made-up price; the local model path end to end against a fake model server; fallback when the local model is down; local model on the same machine.
test_flow.py	1	Full journey from upload to confirmed.

Sample input and output

Chat says	Naive reading	Accord
500 cards 48,000, then 45k, then 46,500 "done"	Last number wins	46,500 agreed, earlier prices kept as history

Chat says	Naive reading	Accord
"cards 600 chahiye" then "54,000 final"	Treats 46,500 and 54,000 as a conflict	54,000 for 600 cards; reason shown: order changed from 500 to 600
"20k advance abhi bhej raha hu"	Could be read as the price	Advance ₹20,000, kept under payment
"GST alag se lagega"	Ignored	Price shown as plus GST
"Advance usual wala", "next week tak"	Accepted as clear	Flagged as vague until a number and a date are given
A quote after acceptance that nobody agrees to	Becomes the price	Earlier accepted price stays; the new quote is flagged

Edge cases we handled

- Amounts that look like phone numbers or years are ignored.
- "500 cards" is a quantity, not ₹500.
- A negated amount ("45k pe nahi ho paega") is not taken as an offer.
- An empty model token, a missing message id, and a model price that is not in the chat are all handled and covered by tests.

Known failures and gaps in testing

- The reader has been tuned on one detailed sample chat and the cases in the tests. We have not measured accuracy on a large set of real chats, so we cannot claim a percentage.
- Very unusual phrasing, other Indian languages in script (for example Kannada in Kannada letters), or sarcasm will be misread by the rules reader. A model may do better, but we have not measured it.
- The local model path is tested against a fake server in the automated tests. Its real-world run on a MacBook was not confirmed when this report was written (section 11).

9. AI, data and external dependencies

AI used to build the project

We used Claude (Anthropic's AI assistant) throughout development: to write and revise code, to debug, to draft the documentation, and to help design the extraction logic and the interface. The team directed the work, chose the problem and the design direction, tested the result and decided what to keep.

AI inside the product

Piece	Use
Rules reader	No AI. Plain Python and regular expressions. Used for the public demo, and as the fallback and the reference for checking models.
Local model (optional)	qwen2.5:7b through Ollama, run on the same Mac. Reads the chat and returns the terms as JSON. Output is checked by the safety layer.

Piece	Use
Claude API (optional)	Same job through the Anthropic API when an API key is set. Not needed for the demo.
Speech and screenshot text (optional)	Voice notes can be transcribed with Whisper (OpenAI API) and screenshots read with Claude vision when keys are set. Without keys the user types the text in. Text produced by a machine is labelled as such so people check it.

The prompt used for extraction tells the model to return only JSON in a fixed shape, never invent values, use the latest price the other side accepted, tie a price to its quantity, treat advances as payment terms, flag vague phrases, always check for six standard gaps, and write short friendly nudges in the chat's own language style.

Data

There is no training dataset. The sample chat (samples/wedding_cards_hinglish.txt) was written by us for the demo. All demo accounts and people in the app are fictional.

Libraries and services

Name	Used for
FastAPI, Uvicorn, python-multipart	Web server and file uploads
reportlab, DejaVu fonts, pypdf	Creating PDFs and checking them in tests
httpx	Calls to the local model and in tests
anthropic (optional)	Claude API client
pytest, Playwright	Automated and browser testing
Ollama and the Qwen 2.5 model (optional)	Running a local model. Model is from Alibaba's Qwen team under its own license.
Archivo, JetBrains Mono	Typefaces
Cloudflare Tunnel (optional, development only)	Reaching a model on another laptop during testing

10. User interface and demonstration

The interface uses a single black-and-white style: heavy borders and hard shadows. A solid black tag means done or confirmed, a dashed grey tag means pending, and a striped one means a problem. Colour is not needed to read status.

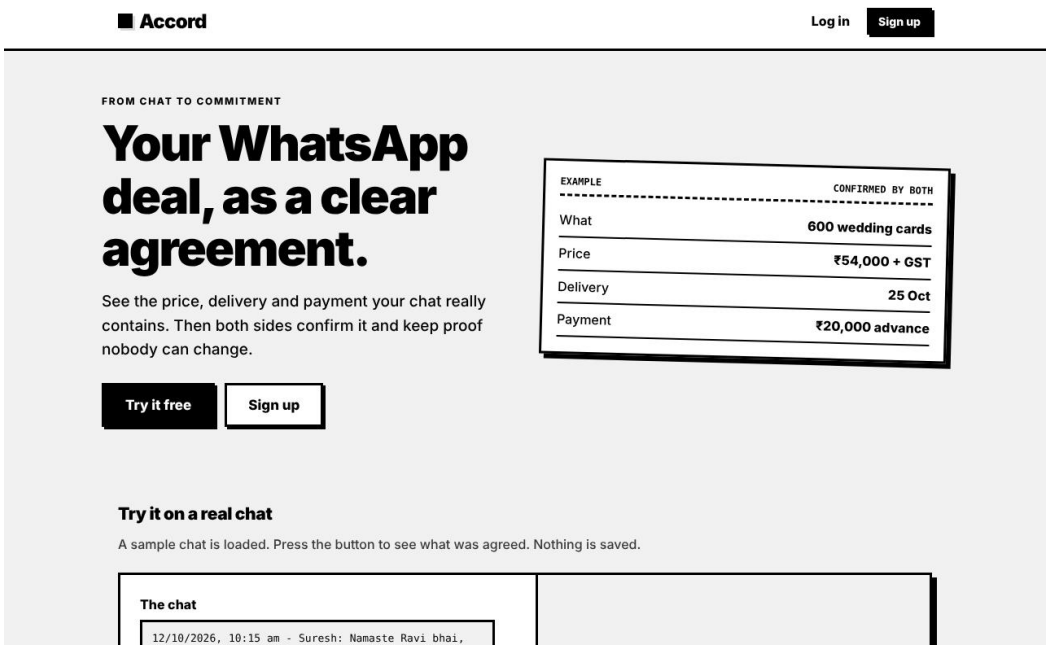


Figure 5. Landing page. Visitors can try the reader on a sample chat without an account.

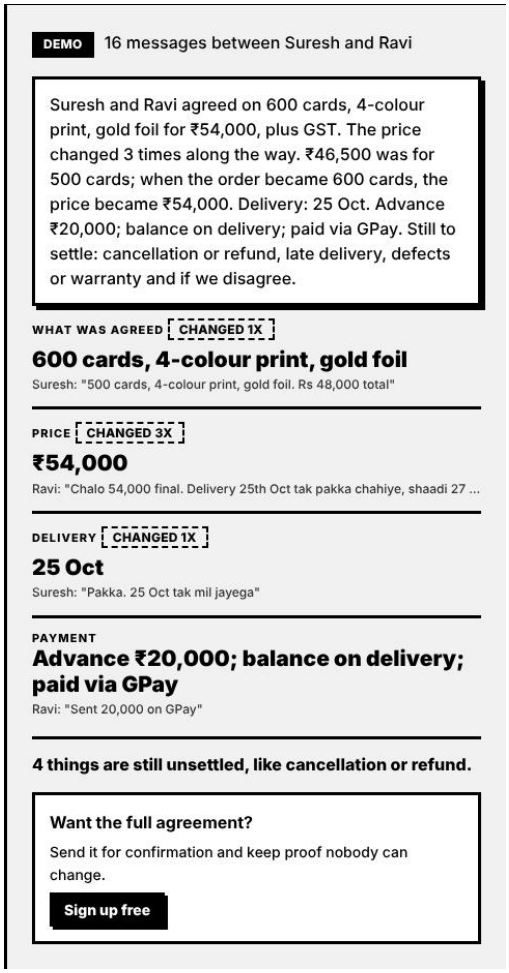


Figure 6. Public demo result for the sample chat: summary, the four terms with the message behind each, and what is still unsettled.

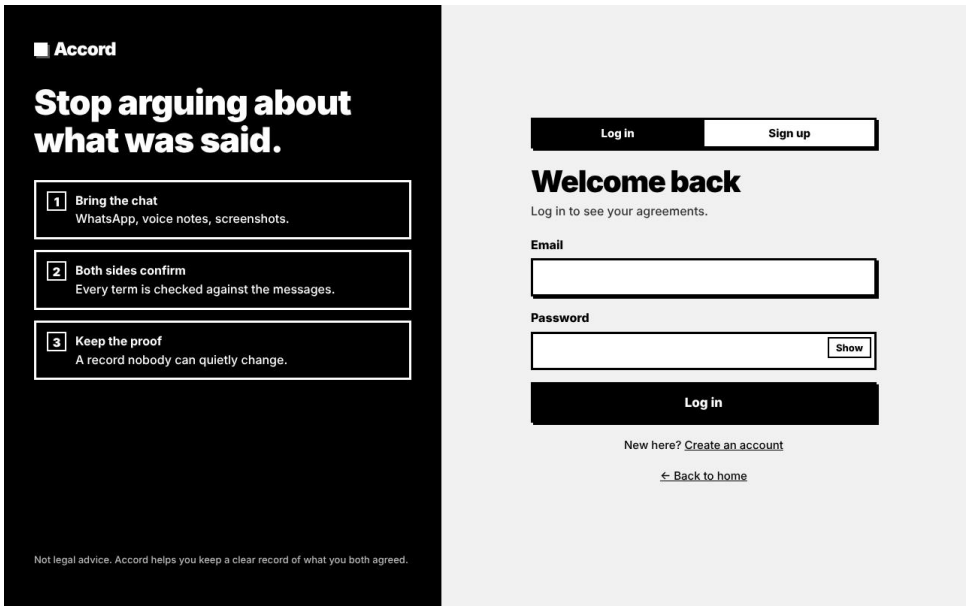


Figure 7. Login.

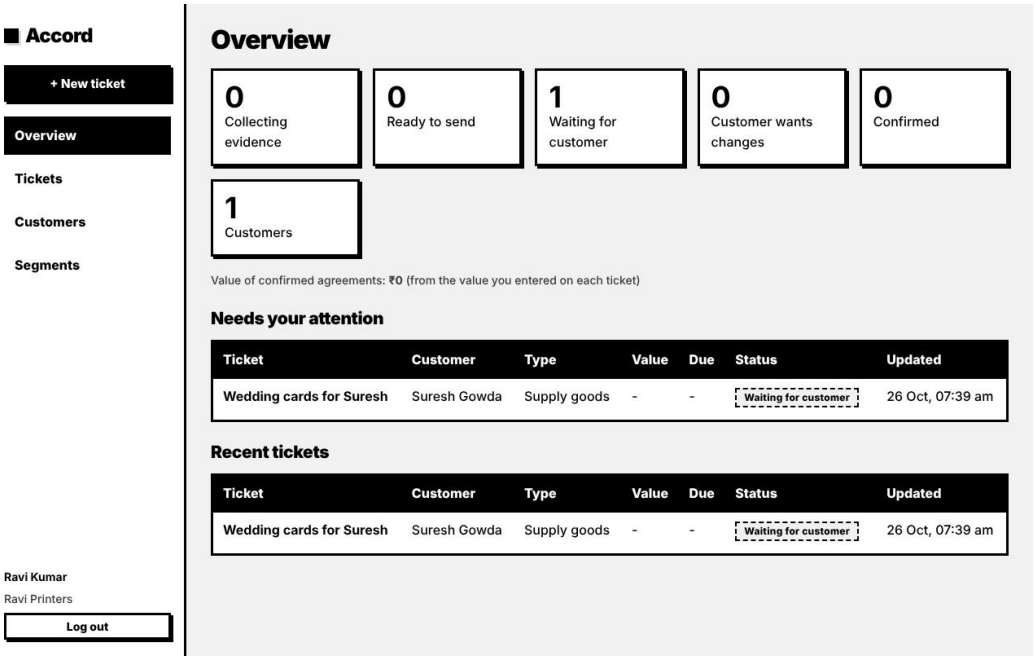


Figure 8. Dealer overview with ticket counts and what needs attention.

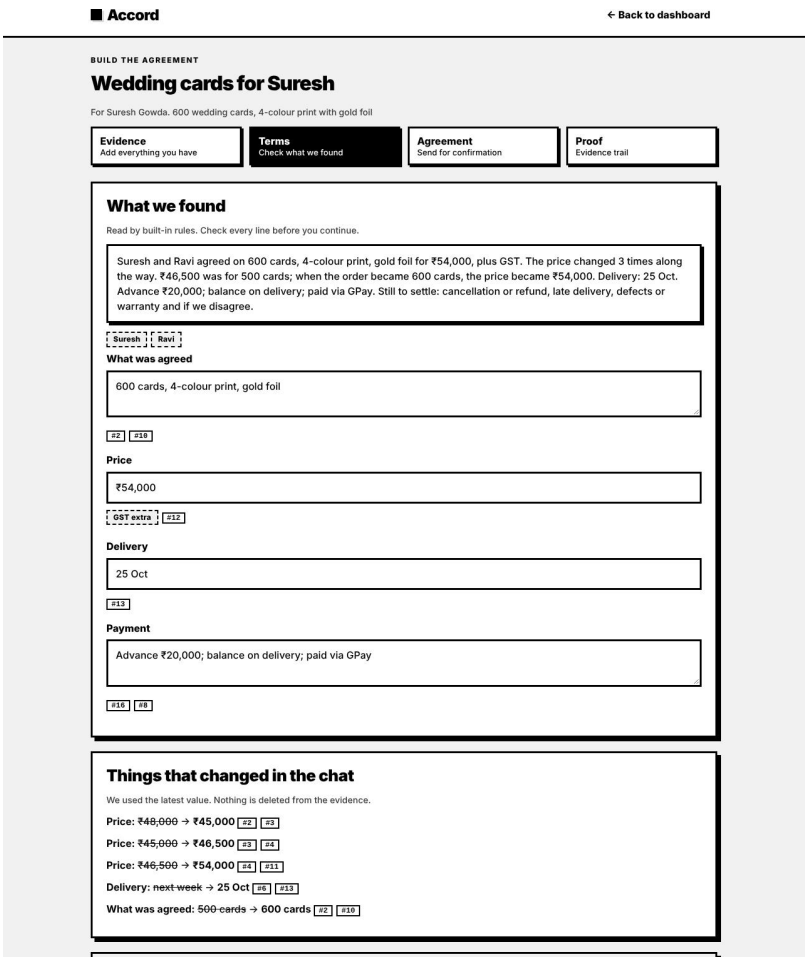


Figure 9. Dealer terms tab: summary, editable terms with message references, what changed along the way, and nudges for unsettled items.

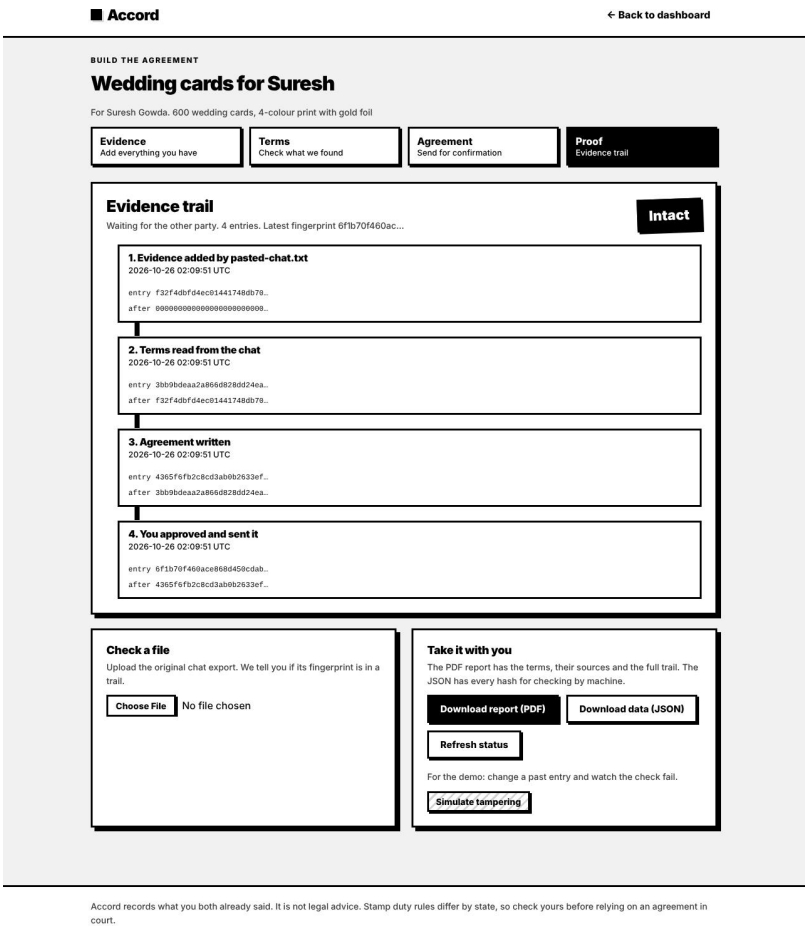


Figure 10. Proof tab: the hash-chained trail, file checker, PDF download and the tamper demo.

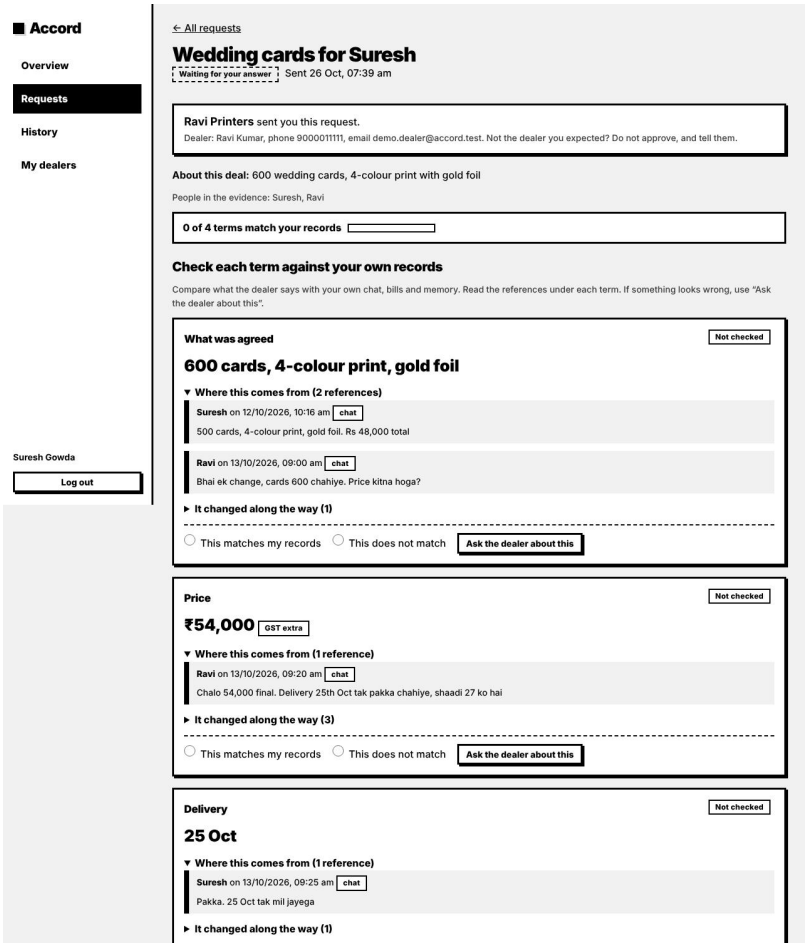


Figure 11. Customer review: each term with only the cited messages, a match or mismatch choice, and approval locked until all terms are checked.

PDF output

The agreement PDF and the evidence report are generated from the same data the screens show. The report includes the terms and their source messages, a table of what changed along the way (with the reason when the quantity changed), the items not yet settled, and the evidence trail.

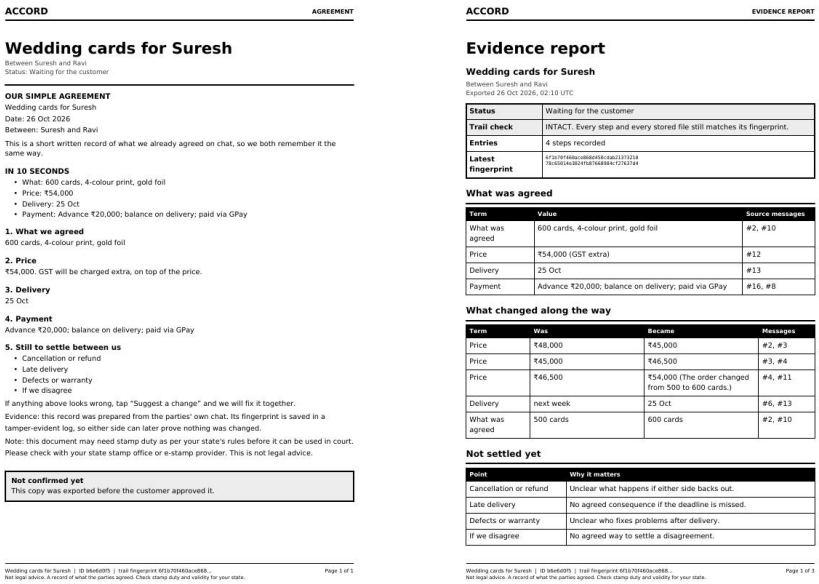


Figure 12. First pages of the agreement PDF (left) and the evidence report PDF (right).

Demo walk-through (about five minutes)

- Landing page: press the button on the sample chat. Point out ₹54,000, the reason for the change from ₹46,500, the term-by-term sources and the list of what is missing.
- Dealer window: open the ticket, show the terms and nudges, then the Proof tab and the tamper demo.
- Customer window (private browser window): open the request, check the terms against the quoted messages, see approval stay locked until all are checked, then approve.
- Back on the dealer side: show the confirmed status and download the PDFs.

11. Limitations and incomplete work

We separate what we demonstrated from what is built but unverified, and from what is only planned.

Demonstrated and tested

Everything in sections 7 and 8, including the rules reader, accounts and invite links, customer verification, evidence chain, PDF export and the fallback behaviour when a model is unreachable.

Built, but not verified on real hardware

- The local model on a MacBook (setup script, model definition, health check). It is covered by automated tests against a fake model server, but at the time of writing we had not confirmed a full run with real Ollama on the target laptop. The first real check is the script that setup_mac.sh runs at the end.
- Voice-note transcription (Whisper) and screenshot reading (Claude vision) need API keys, which we did not use in the final demo. Without keys, text is typed in by the user.

Simulated or partial

- The guest confirmation page uses a one-time code. In demo mode the code is shown on screen instead of being sent by SMS. It shows the flow, but it is not real phone verification.
- Data lives in a single SQLite file. On free hosting tiers it can be wiped on restart, which is why demo accounts can be recreated automatically.

Not built

- Real SMS or WhatsApp integration, and automatic import from WhatsApp (users upload an export).
- Legally recognised e-signature (for example Aadhaar-based or a licensed provider).
- More than two parties in one deal.
- Languages written in their own script (for example Kannada letters) are not read by the rules reader.
- An accuracy benchmark on real-world chats.
- Payment tracking or reminders.

12. Real-world viability and impact

Intended users: small dealers and service providers who sell through chat, and the customers they sell to. The first value for a dealer is a clear record and fewer arguments; for the customer it is knowing exactly what they are agreeing to.

Why it could work

- Nobody has to learn a new habit. The input is the chat they already have.
- The cost to run is low: the rules reader needs no paid service, and the optional model can run on a laptop.
- The customer step takes a minute and is framed as checking, not signing.

Security and privacy

- Passwords are hashed with scrypt, session tokens are stored hashed, and cookies are HttpOnly and SameSite.
- Every route checks the user's role and ownership; tests confirm one dealer cannot read another's data.
- The customer sees only the messages cited behind the terms.
- With the local model, chats do not leave the machine.
- Still to do for real use: rate limits on login, email verification, backups, encryption of stored files and a privacy policy covering stored chats.

Legal and financial limits

Accord is a record-keeping tool, not legal advice. A confirmed Accord agreement shows what both sides said and checked, which helps in a dispute, but whether it is enforceable, and whether stamp duty applies, depends on the state and the kind of deal. We say this on the site. We would want a lawyer's review before telling anyone it can replace a contract. Storing people's chats also brings data protection duties under Indian law that we would need to meet.

Risks

- A wrong extraction could mislead someone who does not check. This is why every term must be checked by both sides, shows its source, and models are checked against the chat.
- A dealer could send a one-sided or edited version. The customer sees when a term was edited after the evidence was read, and the trail records the edit.
- People may not respond. Accord cannot force the other side to confirm.

Scalability

The code is stateless apart from the database and the uploads folder, so moving to Postgres and object storage and running several copies is a straightforward step. The rules reader is cheap enough to run on every request. A model-based reader would need a queue and a worker.

13. Future scope

Next step	Why
Real OTP by SMS and a licensed e-sign option	Turns the confirmation into something stronger than a click.
WhatsApp Business integration	Remove the export step and send nudges directly.
Postgres, object storage, backups	Needed before any real data is stored.
Measure accuracy on many real chats, and improve the rules and the model prompt from the failures	We only have a small set of examples today.
Support Kannada, Hindi and other scripts	Large part of the real user base writes in their own script.
Reminders for delivery and payment dates found in the agreement	The terms are already structured, so the reminders are small to add.
Lawyer review of templates and stamp-duty guidance by state	To say honestly what the record is worth.
Multi-party deals and amendments	Real deals often involve more than two people and change after confirmation.

14. Setup and reproducibility

Run on a Mac with the local model

From the project folder:

```
bash setup_mac.sh
```

```
bash run_mac.sh
```

The first command installs the Python packages and Ollama, downloads the model (about 4.7 GB, once), builds the accord-reader model and tests it on the sample chat. The second starts the app at http://127.0.0.1:8000. Add --fresh to wipe the data and recreate the demo accounts.

Run without any AI

```
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
```

```
python scripts/seed_demo.py
```

```
python -m uvicorn app.main:app --port 8000
```

Run the tests with: python -m pytest -q

Settings (environment variables)

Variable	Meaning
LLM_PROVIDER	auto (default), claude, local or rules
ANTHROPIC_API_KEY	Optional. Enables Claude. Never commit it.
LOCAL_LLM_URL, LOCAL_LLM_TOKEN, LOCAL_LLM_KIND	Optional. Point to a model on another machine. With LLM_PROVIDER=local and no URL, Ollama on the same machine is used.
LOCAL_LLM_MODEL, LOCAL_LLM_TIMEOUT	Model name (default accord-reader) and timeout in seconds
DEMO_MODE	1 shows the one-time code on screen instead of sending an SMS
SEED_DEMO	1 creates the demo accounts on start (used on hosting)
DB_PATH, UPLOAD_DIR	Where the database and uploaded files are stored

Demo logins (fictional, local only)

Role	Email	Password
Dealer (Ravi Printers)	demo.dealer@accord.test	demo12345
Customer (Suresh Gowda)	demo.customer@accord.test	demo12345

These accounts exist only on a freshly seeded local copy. No keys or real credentials are included in the project. To use two roles at once, use one normal browser window and one private window.

15. Team contributions and references

Member	College	Role
Chinmay S	RV University	Backend: server, database, accounts, evidence chain, extraction logic, PDF export, testing and deployment.
Bhakti Chethan	BMS	UI and UX design.
Ayush Iyer	BMS	AI integration.
Ayush Prashanth	BMS	User interface and the landing page.

We used Claude as a coding and writing assistant across all areas of the build, as described in section 9.

References and acknowledgements

- Hackathon problem statement LS-06, From Handshake to Contract (Informal Agreements).
- Documentation for FastAPI, SQLite, reportlab, Ollama and Playwright.
- Qwen 2.5 model by Alibaba's Qwen team, run through Ollama.

- Existing tools we looked at for comparison: PakkaWork (WhatsApp chat to freelance contract with OTP signing), which is listed at hunted.space/product/pakkawork. We only reviewed its public listing, not the product itself.